Event Management Module - Quick Start Guide

Table of Contents

1. Installation
2. First Event in 5 Minutes
3. Common Workflows
4. API Examples
5. Next Steps

Installation

Prerequisites Check

```
# Check .NET version (requires 8.0+)
dotnet --version

# Check PostgreSQL connection
psql -h srv792437.hstgr.cloud -p 5433 -U your_user -d MahamDB -c "SELECT
version();"
```

Setup in 3 Steps

1. Clone and Build

```
cd F:\mahamBackendNET\backend
dotnet restore
dotnet build ZRAdminMahamBackend.sln
```

2. Configure Database

Edit `ZR.Admin.WebApi/appsettings.json`:

```
{
  "dbConfigs": [{
    "ConfigId": "0",
    "DbType": 4,
    "ConnectionString":
"Host=srv792437.hstgr.cloud;Port=5433;Database=MahamDB;Username=your_user;Passw
ord=your_pass"
  }],
```

```
  "InitDb": true  // First time only
}
```

3. Run the Application

```
cd ZR.Admin.WebApi
dotnet run
```

Access the API:

- **Scalar UI**: http://localhost:8887/scalar
- **Swagger**: http://localhost:8887/swagger

First Event in 5 Minutes

Step 1: Get Authentication Token (2 minutes)

```
# Login to get JWT token
curl -X POST "http://localhost:8887/api/login" \
  -H "Content-Type: application/json" \
  -d '{
    "userName": "admin",
    "password": "admin123"
  }'
```

Response:

```
{
  "code": 200,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

Save the token:

```
export TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Step 2: Create Your First Event (1 minute)

```
curl -X POST "http://localhost:8887/event/Event" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "eventCode": "EXPO2025",
    "eventName": "Tech Expo 2025",
    "eventNameAr": "معرض التقنية 2025",
    "eventType": "EXHIBITION",
    "status": "planned",
    "startDate": "2025-06-01T09:00:00Z",
    "endDate": "2025-06-05T18:00:00Z",
    "locationName": "Riyadh Exhibition Center",
    "city": "Riyadh",
    "country": "Saudi Arabia",
    "maxCapacity": 10000,
    "isActive": true
  }'
```

Success Response:

```
{
  "code": 200,
  "msg": "Success",
  "data": {
    "eventId": "1847567890123456789",
    "eventCode": "EXPO2025",
    "eventName": "Tech Expo 2025"
  }
}
```

Step 3: Register a Visitor (1 minute)

```
curl -X POST "http://localhost:8887/event/EventVisitor" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "eventId": "1847567890123456789",
    "visitorType": "GENERAL",
    "firstName": "Ahmed",
    "lastName": "Ali",
    "firstNameAr": "أحمد",
    "lastNameAr": "علي",
    "email": "ahmed.ali@example.com",
    "phone": "+966501234567",
    "status": "registered",
    "isVIP": false
  }'
```

Step 4: Issue a Ticket (1 minute)

```
curl -X POST "http://localhost:8887/event/EventTicket" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "eventId": "1847567890123456789",
    "visitorId": "1847567890123456790",
    "ticketType": "STANDARD",
    "ticketNumber": "TICKET-001",
    "price": 100.00,
    "issueDate": "2025-05-01T10:00:00Z",
    "validFrom": "2025-06-01T09:00:00Z",
    "validTo": "2025-06-05T18:00:00Z",
    "status": "issued",
    "canBeTransferred": false
  }'
```

Done! You now have:

- ☑ An active event
- ☑ A registered visitor
- ☑ An issued ticket

Common Workflows

Workflow 1: Event Creation and Setup

```
# 1. Create event
POST /event/Event

# 2. Create zones for the event
POST /event/EventZone
{
  "eventId": "EVENT_ID",
  "zoneName": "Main Exhibition Hall",
  "zoneType": "EXHIBITION_AREA",
  "maxCapacity": 5000
}

# 3. Create entry gates
POST /event/EventGate
{
  "eventId": "EVENT_ID",
  "gateName": "Main Entrance",
  "gateType": "MAIN_ENTRANCE",
```

```
  "maxCapacityPerHour": 500
}

# 4. Create booths
POST /event/EventBooth
{
  "eventId": "EVENT_ID",
  "boothCode": "A-101",
  "boothName": "Booth A-101",
  "boothType": "STANDARD",
  "sizeSqm": 20,
  "rentalPrice": 5000
}
```

Workflow 2: Visitor Registration and Check-in

```
# 1. Register visitor
POST /event/EventVisitor
{
  "eventId": "EVENT_ID",
  "firstName": "Sara",
  "lastName": "Ahmed",
  "email": "sara@example.com",
  "phone": "+966501234567",
  "visitorType": "GENERAL"
}

# 2. Issue ticket
POST /event/EventTicket
{
  "eventId": "EVENT_ID",
  "visitorId": "VISITOR_ID",
  "ticketType": "STANDARD",
  "price": 100
}

# 3. Check in visitor (on event day)
POST /event/EventVisitor/checkin/VISITOR_ID

# 4. Check out visitor (when leaving)
POST /event/EventVisitor/checkout/VISITOR_ID
```

Workflow 3: Booth Rental Process

```
# 1. Register tenant company
POST /event/EventTenant
{
  "companyName": "Tech Solutions Ltd",
```

```
  "commercialRegNumber": "1234567890",
  "contactName": "Mohammed Ali",
  "email": "contact@techsolutions.com",
  "phone": "+966501234567"
}

# 2. Create rental contract
POST /event/EventContract
{
  "eventId": "EVENT_ID",
  "tenantId": "TENANT_ID",
  "boothId": "BOOTH_ID",
  "contractNumber": "CONTRACT-2025-001",
  "contractDate": "2025-05-01",
  "startDate": "2025-06-01",
  "endDate": "2025-06-05",
  "totalAmount": 5000
}

# 3. Generate invoice
POST /event/EventInvoice
{
  "eventId": "EVENT_ID",
  "contractId": "CONTRACT_ID",
  "tenantId": "TENANT_ID",
  "invoiceNumber": "INV-2025-001",
  "invoiceDate": "2025-05-01",
  "dueDate": "2025-05-15",
  "subtotalAmount": 5000,
  "vatAmount": 750,
  "totalAmount": 5750
}

# 4. Record payment
POST /event/EventPayment
{
  "eventId": "EVENT_ID",
  "invoiceId": "INVOICE_ID",
  "paymentNumber": "PAY-2025-001",
  "paymentDate": "2025-05-10",
  "amount": 5750,
  "paymentMethod": "BANK_TRANSFER",
  "status": "completed"
}
```

Workflow 4: Incident Reporting and Resolution

```
# 1. Report incident
POST /event/EventIncident
{
```

```
  "eventId": "EVENT_ID",
  "incidentNumber": "INC-2025-001",
  "incidentType": "TECHNICAL",
  "title": "Projector not working",
  "description": "Main hall projector is not displaying",
  "priority": "high",
  "severity": "medium",
  "status": "reported",
  "incidentTime": "2025-06-01T14:30:00Z",
  "locationDescription": "Main Hall - Stage Area"
}

# 2. Assign to staff
PUT /event/EventIncident
{
  "incidentId": "INCIDENT_ID",
  "assignedToId": "STAFF_USER_ID",
  "status": "assigned"
}

# 3. Resolve incident
POST /event/EventIncident/resolve/INCIDENT_ID
{
  "resolvedTime": "2025-06-01T15:00:00Z",
  "responseTimeMinutes": 30,
  "status": "resolved"
}
```

API Examples

Query Events with Filters

```
# Get all active events
curl -X GET "http://localhost:8887/event/Event/list?
isActive=true&pageNum=1&pageSize=10" \
  -H "Authorization: Bearer $TOKEN"

# Get events by date range
curl -X GET "http://localhost:8887/event/Event/list?beginStartDate=2025-01-
01&endStartDate=2025-12-31" \
  -H "Authorization: Bearer $TOKEN"

# Get events by type
curl -X GET "http://localhost:8887/event/Event/list?eventType=EXHIBITION" \
  -H "Authorization: Bearer $TOKEN"

# Search by name (supports both English and Arabic)
curl -X GET "http://localhost:8887/event/Event/list?eventName=Tech" \
  -H "Authorization: Bearer $TOKEN"
```

Get Event Details

```
# Get full event information
curl -X GET "http://localhost:8887/event/Event/1847567890123456789" \
  -H "Authorization: Bearer $TOKEN"
```

Update Event Status

```
# Change event status to 'active'
curl -X PUT "http://localhost:8887/event/Event" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "eventId": "1847567890123456789",
    "status": "active"
  }'
```

Export Events to Excel

```
# Export all events
curl -X GET "http://localhost:8887/event/Event/export" \
  -H "Authorization: Bearer $TOKEN" \
  -o events_export.xlsx

# Export filtered events
curl -X GET "http://localhost:8887/event/Event/export?
eventType=CONFERENCE&isActive=true" \
  -H "Authorization: Bearer $TOKEN" \
  -o conferences_export.xlsx
```

Import Events from Excel

```
# 1. Download import template
curl -X GET "http://localhost:8887/event/Event/importTemplate" \
  -o event_import_template.xlsx

# 2. Fill in the template with your data

# 3. Import the file
curl -X POST "http://localhost:8887/event/Event/importData" \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@events_to_import.xlsx"
```

Get Visitors for an Event

```
# Get all visitors for event
curl -X GET "http://localhost:8887/event/EventVisitor/list?
eventId=1847567890123456789" \
  -H "Authorization: Bearer $TOKEN"

# Get only VIP visitors
curl -X GET "http://localhost:8887/event/EventVisitor/list?
eventId=18475678901234567 89&isVIP=true" \
  -H "Authorization: Bearer $TOKEN"

# Get checked-in visitors
curl -X GET "http://localhost:8887/event/EventVisitor/list?
eventId=1847567890123456789&status=checked_in" \
  -H "Authorization: Bearer $TOKEN"
```

Check-in and Check-out

```
# Check in a visitor
curl -X POST
"http://localhost:8887/event/EventVisitor/checkin/1847567890123456790" \
  -H "Authorization: Bearer $TOKEN"

# Check out a visitor
curl -X POST
"http://localhost:8887/event/EventVisitor/checkout/1847567890123456790" \
  -H "Authorization: Bearer $TOKEN"
```

Validate a Ticket

```
# Validate ticket by scanning QR code
curl -X POST
"http://localhost:8887/event/EventTicket/validate/1847567890123456791" \
  -H "Authorization: Bearer $TOKEN"
```

Financial Reports

```
# Get all invoices for an event
curl -X GET "http://localhost:8887/event/EventInvoice/list?
eventId=1847567890123456789" \
  -H "Authorization: Bearer $TOKEN"
```

```
# Get pending invoices
curl -X GET "http://localhost:8887/event/EventInvoice/list?
eventId=1847567890123456789&status=pending" \
  -H "Authorization: Bearer $TOKEN"

# Get all payments for an event
curl -X GET "http://localhost:8887/event/EventPayment/list?
eventId=1847567890123456789" \
  -H "Authorization: Bearer $TOKEN"

# Get payments by method
curl -X GET "http://localhost:8887/event/EventPayment/list?
paymentMethod=BANK_TRANSFER" \
  -H "Authorization: Bearer $TOKEN"
```

Generate PDFs

```
# Generate ticket PDF
curl -X GET "http://localhost:8887/event/EventPdf/ticket/1847567890123456791" \
  -H "Authorization: Bearer $TOKEN" \
  -o ticket.pdf

# Generate permit PDF
curl -X GET "http://localhost:8887/event/EventPdf/permit/1847567890123456792" \
  -H "Authorization: Bearer $TOKEN" \
  -o permit.pdf

# Generate invoice PDF
curl -X GET "http://localhost:8887/event/EventPdf/invoice/1847567890123456793"
\
  -H "Authorization: Bearer $TOKEN" \
  -o invoice.pdf

# Generate contract PDF
curl -X GET "http://localhost:8887/event/EventPdf/contract/1847567890123456794"
\
  -H "Authorization: Bearer $TOKEN" \
  -o contract.pdf
```

Get Lookup Data

```
# Get all event types
curl -X GET "http://localhost:8887/event/EventType/list" \
  -H "Authorization: Bearer $TOKEN"

# Get all visitor types
curl -X GET "http://localhost:8887/event/VisitorType/list" \
  -H "Authorization: Bearer $TOKEN"
```

```
# Get all booth types
curl -X GET "http://localhost:8887/event/BoothType/list" \
  -H "Authorization: Bearer $TOKEN"

# Get all payment methods
curl -X GET "http://localhost:8887/event/PaymentMethod/list" \
  -H "Authorization: Bearer $TOKEN"
```

Next Steps

1. Explore the Full Documentation

Read the comprehensive guide:

```
cat docs/EVENT_MANAGEMENT_GUIDE.md
```

2. Set Up Lookup Tables

Populate your lookup tables with seed data (see full guide for SQL scripts):

- Event types (conference, exhibition, festival, etc.)
- Zone types (main hall, VIP lounge, parking, etc.)
- Gate types (main entrance, VIP entrance, emergency exit, etc.)
- Visitor types (general, VIP, press, exhibitor, staff)
- Ticket types (standard, VIP, early bird, group, student)
- Booth types (standard, premium, corner, island)
- Incident types (medical, security, technical, complaint)
- Payment methods (cash, card, bank transfer, online)

3. Configure Permissions

Set up role-based access control for your users:

- Admin: Full access to all EventManagement features
- Event Manager: Manage events, visitors, tickets
- Finance Staff: Manage invoices and payments only
- Security Staff: Check-in/check-out visitors, validate tickets
- Read-only: View reports and statistics

4. Test with Real Data

Create a test event scenario:

1. Create an event for next month
2. Set up zones and gates

3. Create 10-20 booths
4. Register 5 tenant companies
5. Create contracts and invoices
6. Register 50-100 visitors
7. Issue tickets
8. Simulate check-ins
9. Report and resolve incidents
10. Generate reports and PDFs

5. Integrate with Frontend

If you're building a frontend application:

- Use the API endpoints documented above
- Implement authentication with JWT tokens
- Create forms for event creation, visitor registration, etc.
- Display lists with pagination
- Add filtering and search capabilities
- Implement QR code scanning for check-ins
- Generate and display PDF documents

6. Monitor and Optimize

- Enable logging for API requests
- Monitor database query performance
- Add indexes for frequently searched fields
- Set up alerts for incidents
- Track event occupancy in real-time
- Generate analytics dashboards

7. Deployment

When ready for production:

1. Update `appsettings.Production.json` with production database
2. Set `InitDb: false`
3. Build Docker image
4. Deploy with Docker Compose
5. Set up SSL certificates
6. Configure domain name
7. Set up database backups
8. Monitor application health

Useful Commands

Development

```
# Run in watch mode (auto-reload on changes)
cd ZR.Admin.WebApi
dotnet watch run

# Run specific environment
dotnet run --environment Development
dotnet run --environment Production

# Build release version
dotnet build --configuration Release

# Publish for deployment
dotnet publish --configuration Release --output ./publish
```

Docker

```
# Build image
docker build -t zr-event-management -f ZR.Admin.WebApi/Dockerfile .

# Run container
docker run -d -p 8887:8887 --name event-api zr-event-management

# View logs
docker logs -f event-api

# Stop container
docker stop event-api

# Remove container
docker rm event-api
```

Database

```
# Connect to database
psql -h srv792437.hstgr.cloud -p 5433 -U your_user -d MahamDB

# List all EventManagement tables
\dt event_*

# Count records in tables
SELECT 'event_events' as table_name, COUNT(*) FROM event_events
UNION ALL
SELECT 'event_visitors', COUNT(*) FROM event_visitors
UNION ALL
SELECT 'event_tickets', COUNT(*) FROM event_tickets;

# Check event statistics
```

```
SELECT
  status,
  COUNT(*) as count,
  SUM(current_occupancy) as total_occupancy,
  AVG(max_capacity) as avg_capacity
FROM event_events
GROUP BY status;
```

Quick Reference Card

Most Used Endpoints

Action	Method	Endpoint	Body Required
List events	GET	`/event/Event/list`	No
Create event	POST	`/event/Event`	Yes
Update event	PUT	`/event/Event`	Yes
Delete event	POST	`/event/Event/delete/{ids}`	No
Register visitor	POST	`/event/EventVisitor`	Yes
Check-in	POST	`/event/EventVisitor/checkin/{id}`	No
Issue ticket	POST	`/event/EventTicket`	Yes
Validate ticket	POST	`/event/EventTicket/validate/{id}`	No
Create booth	POST	`/event/EventBooth`	Yes
Register tenant	POST	`/event/EventTenant`	Yes
Create contract	POST	`/event/EventContract`	Yes
Create invoice	POST	`/event/EventInvoice`	Yes
Record payment	POST	`/event/EventPayment`	Yes
Report incident	POST	`/event/EventIncident`	Yes

Status Values

Event Status:

- `planned` - Event is being planned
- `active` - Event is currently running
- `completed` - Event has finished
- `cancelled` - Event was cancelled

Visitor Status:

- `registered` - Visitor registered but not checked in
- `checked_in` - Visitor is currently at the event
- `checked_out` - Visitor has left the event
- `cancelled` - Registration cancelled

Ticket Status:

- `issued` - Ticket issued to visitor
- `validated` - Ticket scanned and validated
- `used` - Ticket has been used
- `cancelled` - Ticket cancelled
- `expired` - Ticket validity expired

Booth Rental Status:

- `available` - Booth is available for rent
- `reserved` - Booth reserved but contract not signed
- `occupied` - Booth is rented and occupied
- `maintenance` - Booth under maintenance

Invoice Status:

- `pending` - Invoice generated, awaiting payment
- `partial` - Partially paid
- `paid` - Fully paid
- `overdue` - Payment overdue
- `cancelled` - Invoice cancelled

Incident Status:

- `reported` - Incident reported
- `assigned` - Assigned to staff
- `in_progress` - Being resolved
- `resolved` - Issue resolved
- `closed` - Incident closed

Support

Need Help?

1. **Documentation**: Read the full guide at `docs/EVENT_MANAGEMENT_GUIDE.md`
2. **API Docs**: Visit http://localhost:8887/scalar for interactive API documentation
3. **Logs**: Check application logs in `ZR.Admin.WebApi/logs/`
4. **Issues**: Report bugs or request features on GitHub

Common Questions

Q: How do I reset the database?
A: Set `InitDb: true` and restart the application. Or use `POST /event/Event/clean` (admin only).

Q: How do I get a list of all permissions?

A: Check `docs/EVENT_MANAGEMENT_GUIDE.md` section "API Endpoints" for all permission strings.

Q: Can I use SQL Server instead of PostgreSQL?

A: Yes, change `DbType` to `1` in `appsettings.json` and update connection string.

Q: How do I add custom event types?

A: Insert into `event_types` table or use the `/event/EventType` API endpoints.

Q: Is Arabic language fully supported?

A: Yes, all entities have Arabic name fields (e.g., `EventNameAr`, `FirstNameAr`).

Happy Event Managing! 🎉

For detailed documentation, see: EVENT_MANAGEMENT_GUIDE.md